CampusGuard: Implementación y Benchmarking de Algoritmos de Búsqueda (BFS, DFS y A*) para la Prevención de la Deserción Académica

Jeison Montoya - 9 de noviembre de 2026

Resumen- En este trabajo se avanza desde el modelado conceptual de la Fase 1 hacia la resolución algorítmica de problemas mediante búsqueda en espacio de estados para el agente racional CampusGuard. Se abstrae el ecosistema universitario hacia un entorno determinista y discreto que evalúa intervenciones preventivas en una ventana crítica de 48 horas. Se implementaron y evaluaron experimentalmente sobre 8 perfiles de estudiantes los algoritmos BFS, DFS y A* (utilizando la base AIMA-Python). Los resultados cuantitativos evidencian que A*, mediante una función heurística admisible y consistente basada en el costo mínimo teórico restante, reduce el costo de intervención en un 30% frente a BFS y un 44% frente a DFS, garantizando completitud y optimalidad.

Palabras clave: Búsqueda en espacio de estados, BFS, DFS, A*, heurística admisible, costo de intervención, deserción académica.

1. Introducción y Retoma del Problema (Fase 1)

En la educación superior, la deserción estudiantil en los primeros semestres alcanza aproximadamente un 18%, siendo atribuible principalmente a la naturaleza reactiva de los apoyos institucionales tradicionales (Tinto, 1993). En la Fase 1 se formuló CampusGuard, un agente racional con arquitectura basada en utilidad que procesa percepciones del LMS, asistencia y calificaciones para activar intervenciones antes de 48 horas. El presente informe formaliza computacionalmente la toma de decisiones de CampusGuard como un problema de búsqueda en espacio de estados, comparando el desempeño de algoritmos ciegos (BFS, DFS) frente a búsqueda informada (A*) para optimizar el uso de recursos institucionales escasos.

2. Abstracción del Entorno

El entorno real caracterizado en la Fase 1 es parcialmente observable, estocástico, dinámico y multiagente. Para posibilitar la búsqueda algorítmica clásica (Russell & Norvig, 2021), se abstrae a un marco determinista y discreto:

Dimensión Original	Abstracción Búsqueda	Justificación Operativa
Parcialmente observable	Totalmente observable	Estado condensa historial LMS, asistencia y notas en un vector.
Estocástico		

3. Formalización del Espacio de Estados

El problema se formaliza mediante la quintupla clásica P = <s_0, A, T, Goal, c> conforme a Russell y Norvig (2021):

- **Estado (s):** Tupla s = <P, I, h>, donde P es el perfil del estudiante (score_riesgo en [0, 100]), I es la secuencia de intervenciones aplicadas, y h en [0, 48] es la hora acumulada en la ventana de atención.
- **Estado Inicial (s_0):** s_0 = <P_inicial, (), 0>, donde P_inicial tiene score_riesgo > 25 (nivel medio, alto o crítico), sin intervenciones previas e inicio en hora h = 0.
- **Conjunto de Acciones A(s):** Intervenciones válidas no aplicadas previamente que cumplen h + duracion(a) <= 48 y |I| < 4: (1) MONITOREAR (costo=0.5h, dur=1h, -5% riesgo); (2) NOTIFICAR_BIENESTAR (costo=1.0h, dur=2h, -15%); (3) RECOMENDAR_TUTORIA (costo=3.0h, dur=8h, -25%); y (4) REDISTRIBUIR_CARGA (costo=5.0h, dur=12h, -35%). Factor de ramificación b <= 3.
- **Modelo de Transición T(s, a):** Transición determinista s' = <P', I U {a}, h + dur(a)>, donde score_riesgo(P') = max(0, score_riesgo(P) - efectividad(a) * 100).
- **Prueba de Meta Goal(s):** Goal(s) <=> P.score_riesgo <= 25 (nivel BAJO de deserción) y h <= 48 horas.
- **Función de Costo c(s, a, s'):** c(s, a, s') = costo(a) en horas-equivalentes de talento institucional. Costo acumulado de ruta: g(n) = Sum c(s_i, a_i, s_i+1), el cual se desea minimizar.

4. Implementación de Búsquedas No Informadas (BFS y DFS)

Se implementaron los algoritmos adaptando la base estándar de AIMA-Python, extendiendo la clase Problem en CampusGuardProblem. BFS (Breadth-First Search) implementa exploración por niveles mediante una cola FIFO y conjunto cerrado de estados explorados. Garantiza encontrar el camino con menor número de pasos, pero es ciego respecto a costos de aristas no homogéneos. DFS (Depth-First Search) utiliza una pila LIFO explorando recursivamente en profundidad; consume un espacio lineal mínimo, pero carece de garantías de optimalidad y puede seleccionar intervenciones innecesariamente costosas.

5. Diseño de Búsqueda Informada (A*) y Heurística Admisible

A* evalúa nodos mediante f(n) = g(n) + h(n), donde g(n) es el costo real consumido y h(n) es la estimación del costo mínimo requerido para alcanzar un score de riesgo <= 25. Se formuló la heurística de Costo Mínimo Teórico Restante:

h(n) = max(0, score_riesgo(n) - 25) / max_{a en A_disp} [(efectividad(a) * 100) / costo(a)]

Donde A_disp son las intervenciones aplicables que caben en el tiempo remanente (48 - h). La máxima ratio de efectividad por hora la ostenta NOTIFICAR_BIENESTAR (15 pts / 1.0 h = 15.0 pts/h) y RECOMENDAR_TUTORIA (25 pts / 3.0 h = 8.33 pts/h).

- **Demostración de Admisibilidad (h(n) <= h*(n)):** h(n) relaja tres restricciones del problema: (1) supone que todo el score restante se mitiga a la máxima ratio de eficiencia teórica; (2) ignora la restricción de 48 h; y (3) asume divisibilidad de acciones sin penalizar el límite de 4 intervenciones. Dado que ninguna combinación de acciones reales puede superar la mejor ratio teórica, h(n) jamás

6. Ejecución y Registro de Experimentos (Benchmark)

Se ejecutaron BFS, DFS y A* sobre un conjunto controlado de 8 perfiles estudiantiles con riesgos iniciales entre 27.4 y 74.1. Los tres algoritmos partieron del mismo estado inicial s_0, evaluando exhaustivamente nodos expandidos, nodos generados, tiempo de CPU en milisegundos y costo final de la solución g(n).

Tabla 1. Benchmark Comparativo Cuantitativo: BFS vs. DFS vs. A* en CampusGuard

Estudiante	Riesgo	Algoritmo	Éxito	N. Exp.	N. Gen.	Tiempo(ms)	Costo (g)
EST-2024-003	33.4 (Med)	BFS	SÍ	1	4	0.14	3.00 h
EST-2024-003	33.4 (Med)	DFS	SÍ	1	4	0.10	5.00 h
EST-2024-003	33.4 (Med)	A*	SÍ	2	7	0.20	1.00 h (Ópt)
EST-2024-001	46.4 (Med)	BFS	SÍ	1	4	0.07	3.00 h
EST-2024-001	46.4 (Med)	DFS	SÍ	1	4	0.07	5.00 h
EST-2024-001	46.4 (Med)	A*	SÍ	5	14	0.39	3.00 h (Ópt)
EST-2024-002	74.1 (Alt)	BFS	SÍ	3	10	0.20	8.00 h
EST-2024-002	74.1 (Alt)	DFS	SÍ	2	7	0.13	6.00 h
EST-2024-002	74.1 (Alt)	A*	SÍ	17	34	1.42	6.00 h (Ópt)
EST-2024-004	55.9 (Alt)	BFS	SÍ	1	4	0.08	5.00 h
EST-2024-004	55.9 (Alt)	DFS	SÍ	1	4	0.07	5.00 h
EST-2024-004	55.9 (Alt)	A*	SÍ	8	21	0.61	4.00 h (Ópt)
EST-2024-005	43.1 (Med)	BFS	SÍ	1	4	0.07	3.00 h
EST-2024-005	43.1 (Med)	DFS	SÍ	1	4	0.07	5.00 h
EST-2024-005	43.1 (Med)	A*	SÍ	3	10	0.25	1.50 h (Ópt)
EST-2024-006	67.4 (Alt)	BFS	SÍ	3	10	0.20	8.00 h
EST-2024-006	67.4 (Alt)	DFS	SÍ	2	7	0.13	6.00 h
EST-2024-006	67.4 (Alt)	A*	SÍ	10	25	0.79	4.50 h (Ópt)
EST-2024-007	27.4 (Med)	BFS	SÍ	1	4	0.06	0.50 h
EST-2024-007	27.4 (Med)	DFS	SÍ	1	4	0.07	5.00 h
EST-2024-007	27.4 (Med)	A*	SÍ	1	4	0.09	0.50 h (Ópt)
EST-2024-008	49.6 (Med)	BFS	SÍ	1	4	0.07	3.00 h
EST-2024-008	49.6 (Med)	DFS	SÍ	1	4	0.08	5.00 h
EST-2024-008	49.6 (Med)	A*	SÍ	5	14	0.37	3.00 h (Ópt)

Tabla 2. Resumen Comparativo de Promedios Globales (8 perfiles)

Algoritmo	% Éxito	Nodos Exp. Prom.	Tiempo Prom. (ms)	Costo Prom. (h)
BFS	100.0 %	1.5	0.11 ms	4.19 h
DFS	100.0 %	1.2	0.09 ms	5.25 h
A* (Heurístico)	100.0 %	6.4	0.52 ms	2.94 h (Ahorro 30-44%)

Análisis del Benchmark: La tasa de éxito fue del 100% en los tres algoritmos, confirmando que la ventana de 48 horas y el catálogo de acciones ofrecen solución viable para todos los perfiles de riesgo. Los tiempos de ejecución en los tres casos son inferiores a 1.5 ms, demostrando alta viabilidad computacional para su ejecución en tiempo real en sistemas universitarios.

7. Análisis de Complejidad Computacional (Notación Big O)

El análisis formal compara los requerimientos asintóticos de tiempo y espacio frente a las restricciones operativas de CampusGuard: factor de ramificación b <= 3 (acciones válidas), profundidad óptima d en [1, 3] y profundidad máxima m = 4:

Algoritmo	Complejidad T.	Complejidad E.	Optimalidad	CampusGuard (Nodos Máx)
BFS	O(b^d)	O(b^d)	Solo costo uniforme	Hasta b^2 = 9 nodos (~13 total)
DFS	O(b^m)	O(b * m) = O(m)	No óptimo	Hasta Sum b^i = 121 nodos (peor caso)
A*	O(b^d) / O(b^(d/eps))	O(b^d)	Óptimo (h admisible)	5 a 17 nodos guiados por f(n)

- **Sustentación Temporal O(b^d) vs O(b^m):** BFS explora exhaustivamente todos los nodos nivel a nivel, requiriendo en el peor caso O(b^d) operaciones elementales. DFS desciende a lo largo de ramas arbitrarias hasta agotar el límite m = 4, con una complejidad de O(b^m); en árboles profundos o sin poda de ciclos, DFS resulta inmanejable, si bien en CampusGuard el espacio finito acota el peor caso a 121 estados.
- **Sustentación Espacial O(b^d) vs O(b*m):** El cuello de botella crítico de BFS es la memoria: almacena en la frontera todos los nodos del nivel d (O(b^d)), lo que en dominios extensos colapsa la memoria. DFS almacena únicamente el camino activo desde la raíz junto con los hermanos no explorados en cada nivel, logrando un consumo lineal O(b * m), altamente eficiente.
- **Comportamiento de A*:** A* retiene la frontera en una cola de prioridad, compartiendo la cota espacial O(b^d) con BFS. Sin embargo, su complejidad temporal depende de la calidad de h(n): con una heurística admisible y consistente, A* poda ramas subóptimas y restringe la búsqueda a un túnel directo hacia la solución de costo mínimo.

8. Evaluación Crítica de la Heurística en A*

Los resultados de la Tabla 2 reflejan un hallazgo esencial de ingeniería de software e inteligencia artificial: A* expandió en promedio 6.4 nodos frente a 1.5 de BFS, pero alcanzó un costo promedio de solución de 2.94 horas frente a 4.19 horas de BFS (ahorro del 29.8%) y 5.25 horas de DFS (ahorro del 44.0%).

- **Discusión sobre Nodos Expandidos vs. Costo:** En problemas donde todas las acciones tienen costo unitario, BFS encuentra el costo óptimo garantizado al hallar el nodo meta más superficial. Sin embargo, en CampusGuard los costos son dispares (de 0.5 h a 5.0 h). BFS se detiene en la primera acción única que satisface la meta (por ejemplo, REDISTRIBUIR_CARGA, costo 5.0 h, en EST-2024-004), mientras que A* analiza combinaciones más económicas (TUTORÍA + BIENESTAR, costo 4.0 h). A* expande más nodos precisamente para garantizar la optimalidad global.
- **Impacto Práctico Institucional:** A nivel institucional en la Universidad Militar Nueva Granada, la diferencia en tiempo de CPU (0.52 ms vs. 0.11 ms) es imperceptible

9. Código Fuente, Repositorio y Reproducibilidad

El código fuente completo, documentado bajo estándares PEP 8 y modularizado, se encuentra alojado en el repositorio público:

Repositorio GitHub: https://github.com/jeisonmontoya/campusguard-search

- **Estructura Modular del Código:** src/campusguard_problem.py (formalización matemática y subclase Problem de AIMA); src/aima_search.py (implementación de algoritmos BFS, DFS y A* en grafos); src/search_runner.py (ejecución del benchmark, recolección de métricas y exportación JSON/Markdown); y benchmark/ (registros de resultados brutos).
- **Instrucciones de Ejecución:** Para replicar los experimentos: clonar el repositorio, situarse en el directorio raíz o en src y ejecutar 'python search_runner.py'. No requiere dependencias de terceros (utiliza exclusivamente la librería estándar de Python 3.8+).

10. Conclusiones

1. La abstracción formal del problema CampusGuard permitió transformar un entorno institucional complejo en un espacio de búsqueda determinista y discreto completamente resoluble dentro de la ventana crítica de 48 horas.
2. La implementación computacional de BFS, DFS y A* sobre la base canónica de AIMA demostró una tasa de éxito del 100% en la detección y resolución de planes de intervención académica preventiva.
3. Se demostró teórica y empíricamente que la heurística de costo mínimo teórico restante es admisible y consistente, garantizando formalmente la optimalidad de A*.
4. Frente a acciones con costos heterogéneos, A* demostró ser el algoritmo superior para la toma de decisiones racionales, reduciendo el gasto de recursos institucionales en hasta un 44% respecto a búsquedas no informadas.

Referencias

Barocas, S., Hardt, M., & Narayanan, A. (2023). Fairness and machine learning: Limitations and opportunities. MIT Press.

Congreso de la República de Colombia. (2012). Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales.

Russell, S., & Norvig, P. (2021). Artificial intelligence: A modern approach (4.ª ed.). Pearson.

Simon, H. A. (1955). A behavioral model of rational choice. The Quarterly Journal of Economics, 69(1), 99-118. https://doi.org/10.2307/1884852

Tinto, V. (1993). Leaving college: Rethinking the causes and cures of student attrition (2.ª ed.). University of Chicago Press.